

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа №5

Выполнил:

Плахтий Марк

К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2025 г.

Задача

- подключить и настроить kafka;
- реализовать межсервисное взаимодействие посредством kafka.

Ход работы

1. Подключение и настройка Kafka

В качестве брокера сообщений был выбран Apache Kafka. Для развертывания всей необходимой инфраструктуры был использован docker-compose (рисунок 1).

```

1  version: '3.8'
2  >> services:
3  >   zookeeper:
4     image: confluentinc/cp-zookeeper:7.4.3
5     environment:
6       ZOOKEEPER_CLIENT_PORT: 2181
7       ZOOKEEPER_TICK_TIME: 2000
8     ports:
9       - "2181:2181"
10
11  >   kafka:
12     image: confluentinc/cp-kafka:7.4.3
13     depends_on:
14       - zookeeper
15     ports:
16       - "9092:9092"
17     environment:
18       KAFKA_BROKER_ID: 1
19       KAFKA_ZOOKEEPER_CONNECT: "zookeeper:2181"
20       KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
21       KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
22       KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
23       KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
24       KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
25
26   networks:
27     default:
28       driver: bridge

```

Рисунок 1 – Настройка kafka в docker-compose

2. Реализация межсервисного взаимодействия

2.1. Код потребителя (consumer)

вот переписанный текст, но можешь его лучше адаптировать

В качестве потребителя (consumer) выступает микросервис для рассылки email-уведомлений о бронировании, написанный на Node.js. Сервис подключается к Kafka, подписывается на топик бронирований и читает сообщения, закодированные в формате JSON.

Происходит десериализация данных, из которых извлекается информация о клиенте и данные бронирования, включая email получателя. Далее по

SMTP-протоколу отправляется письмо с подтверждением бронирования, содержащим детали клиента, номер комнаты, даты, количество гостей и итоговую сумму.

```
import { Kafka } from 'kafkajs';
import nodemailer from 'nodemailer';

const kafka = new Kafka({
  clientId: 'booking-mailer',
  brokers: KAFKA_BROKERS.split(','),
});

const consumer = kafka.consumer({ groupId: 'booking-mails' });

const transporter = nodemailer.createTransport({
  host: MAIL_HOST,
  port: Number(MAIL_PORT),
  secure: false,
  requireTLS: true,
  auth: {
    user: MAIL_USER,
    pass: MAIL_PASS,
  },
  tls: {
    minVersion: 'TLSv1.2',
  },
});

async function sendBookingMail(to: string, booking: any) {
  const info = await transporter.sendMail({
    from: `${MAIL_FROM_NAME} <${MAIL_USER}>`,
    to,
    subject: 'Подтверждение бронирования номера',
    text: `Здравствуйте, ${booking.client.name}!

Ваше бронирование подтверждено:
Номер: ${booking.roomNumber}
Даты: с ${booking.checkInDate} по ${booking.checkOutDate}
Гостей: ${booking.guests}
Сумма: ${booking.totalPrice} руб.

Спасибо за выбор нашего отеля!`,
    html: `<h2>Здравствуйте, ${booking.client.name}</h2>
<p>Ваше бронирование подтверждено:</p>
<p>Номер: ${booking.roomNumber}</p>
<p>Даты: с ${booking.checkInDate} по ${booking.checkOutDate}</p>
<p>Гостей: ${booking.guests}</p>
<p>Сумма: ${booking.totalPrice} руб.</p>
<p>Спасибо за выбор нашего отеля!</p>`
  });
}
```

Рисунок 2

```

    Спасибо за выбор нашего отеля!`,
    html: `<h2>Здравствуйте, ${booking.client.name}!</h2>
    <p>Ваше бронирование подтверждено:</p>
    <ul>
      <li><b>Номер:</b> ${booking.roomNumber}</li>
      <li><b>Даты:</b> с ${booking.checkInDate} по ${booking.checkOutDate}</li>
      <li><b>Гостей:</b> ${booking.guests}</li>
      <li><b>Сумма:</b> ${booking.totalPrice} руб.</li>
    </ul>
    <p>Спасибо за выбор нашего отеля!</p>`
  });
  console.log(`Booking mail sent: ${info.messageId}`);
}

async function run() {
  await consumer.connect();
  await consumer.subscribe({ topic: 'room_bookings', fromBeginning: false });
  console.log('Subscribed to topic room_bookings');
  await consumer.run({
    eachMessage: async ({ message }) => {
      try {
        if (!message.value) return;
        const booking = JSON.parse(message.value.toString());
        await sendBookingMail(booking.client.email, booking);
      } catch (err) {
        console.log('Error sending mail', err);
      }
    }
  });
}

run();

```

Рисунок 3

2.2. Код отправителя (producer)

В качестве отправителя данных (producer) используется сервис бронирования отелей. При определённых событиях в системе (например, при создании нового бронирования) формируется объект с данными бронирования. Далее эти данные сериализуются в формат JSON. После этого сообщение отправляется в заданный топик Kafka с помощью метода `publish` адаптера `KafkaClient`. Пример кода отправки сообщения через kafka-продюсер приведён выше.

```
class HotelBookingSender {  
  private producer: Producer;  
  
  0 references  
  constructor(kafkaBrokers: string[]) {  
    const kafka = new Kafka({  
      clientId: 'hotel-booking-sender',  
      brokers: kafkaBrokers,  
    });  
    this.producer = kafka.producer();  
  }  
  
  async connect() {  
    await this.producer.connect();  
  }  
  
  async sendBooking(booking: HotelBooking, topicName: string) {  
    const message = {  
      value: JSON.stringify(booking),  
    };  
    await this.producer.send({  
      topic: topicName,  
      messages: [message],  
    });  
  }  
  
  async disconnect() {  
    await this.producer.disconnect();  
  }  
}
```

Рисунок 4

```

export class KafkaClient {
  private readonly kafka: Kafka;
  private producer: Producer | null = null;
  private readonly brokerUrl: string;

  0 references
  constructor(brokerUrl: string) {
    this.brokerUrl = brokerUrl;
    this.kafka = new Kafka({
      clientId: 'hotel-booking-service',
      brokers: [brokerUrl],
    });
  }

  async connect() {
    this.producer = this.kafka.producer();
    await this.producer.connect();
  }

  async publish(queueName: string, message: string) {
    if (!this.producer) {
      throw new Error('Call connect before publish');
    }
    await this.producer.send({
      topic: queueName,
      messages: [{ value: message }],
    });
  }

  async close() {
    if (this.producer) {
      await this.producer.disconnect();
      this.producer = null;
    }
  }
}

```

Рисунок 5

Вывод

В ходе выполнения практической работы была выстроена асинхронная коммуникация между микросервисами с помощью брокера сообщений kafka.